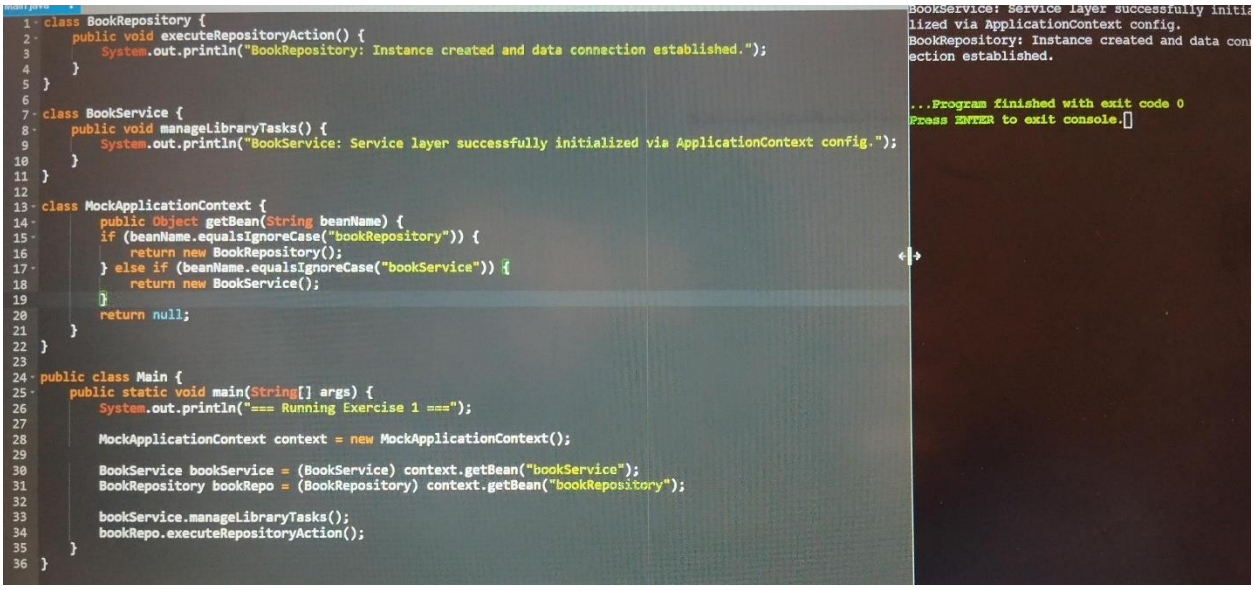
Cognizant Week-2 Deep Skilling Exercises

Spring core and Maven

Exercise 1 : Configuring a Basic Spring Application

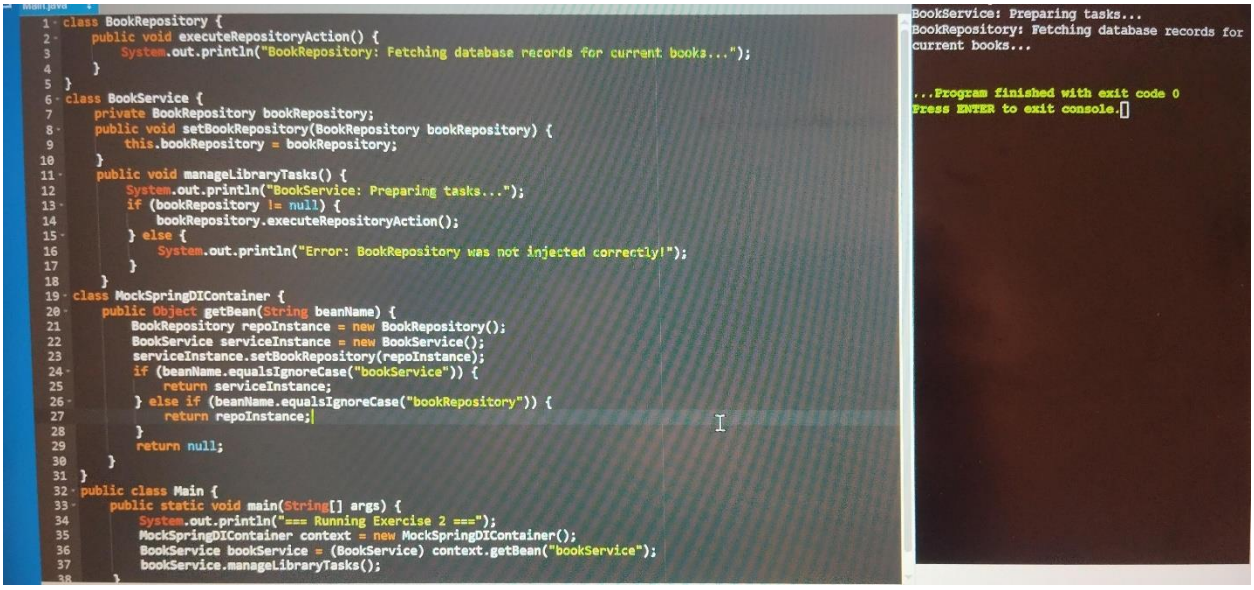


```
class BookRepository {
    public void executeRepositoryAction() {
        System.out.println("BookRepository: Instance created and data connection established.");
    }
}

class BookService {
    public void manageLibraryTasks() {
        System.out.println("BookService: Service layer successfully initialized via ApplicationContext config.");
    }
}

class MockApplicationContext {
        public Object getBean(String beanName) {
        if (beanName.equalsIgnoreCase("bookRepository")) {
            return new BookRepository();
        } else if (beanName.equalsIgnoreCase("bookService")) {
            return new BookService();
        }
        return null;
    }
}

public class Main {
    public static void main(String[] args) {
        System.out.println("=== Running Exercise 1 ===");

        MockApplicationContext context = new MockApplicationContext();

        BookService bookService = (BookService) context.getBean("bookService");
        BookRepository bookRepo = (BookRepository) context.getBean("bookRepository");

        bookService.manageLibraryTasks();
        bookRepo.executeRepositoryAction();
    }
}
```

```
BookService: Service layer successfully initia
lized via ApplicationContext config.
BookRepository: Instance created and data con
ection established.


...Program finished with exit code 0
Press ENTER to exit console.
```

- **Core Objective:** Simulates Spring's **Inversion of Control (IoC) Container** and basic bean retrieval within a single-file online environment.
- **Key Simulated Components:**
    - BookRepository: Acts as the backend data access object (DAO) layer bean.
    - BookService: Acts as the business logic layer bean.
    - MockApplicationContext: Simulates the applicationContext.xml configuration parser by registering, instantiating, and delivering objects on demand via the getBean() method.

Exercise 2 : Implementing Dependency injection



```
class BookRepository {
    public void executeRepositoryAction() {
        System.out.println("BookRepository: Fetching database records for current books...");
    }
}
class BookService {
    private BookRepository bookRepository;
    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }
    public void manageLibraryTasks() {
        System.out.println("BookService: Preparing tasks...");
        if (bookRepository != null) {
            bookRepository.executeRepositoryAction();
        } else {
            System.out.println("Error: BookRepository was not injected correctly!");
        }
    }
class MockSpringDIContainer {
    public Object getBean(String beanName) {
        BookRepository repoInstance = new BookRepository();
        BookService serviceInstance = new BookService();
        serviceInstance.setBookRepository(repoInstance);
        if (beanName.equalsIgnoreCase("bookService")) {
            return serviceInstance;
        } else if (beanName.equalsIgnoreCase("bookRepository")) {
            return repoInstance;
        }
        return null;
    }
}
public class Main {
    public static void main(String[] args) {
        System.out.println("=== Running Exercise 2 ===");
        MockSpringDIContainer context = new MockSpringDIContainer();
        BookService bookService = (BookService) context.getBean("bookService");
        bookService.manageLibraryTasks();
    }
```

```
BookService: Preparing tasks...
BookRepository: Fetching database records for
current books...

...Program finished with exit code 0
Press ENTER to exit console.
```

Core Objective: Demonstrates Setter-based Dependency Injection (DI), showcasing how Spring automatically wires dependencies together so components remain loosely coupled.

Key Simulated Mechanics:

The Setter Method: A explicit setBookRepository(...) method is added to BookService, allowing external forces to inject the repository layer.

The Wiring Engine: MockSpringDIContainer acts as the advanced IoC engine. It instantiates the repository first, passes it directly into

the service via the setter method, and caches the fully wired component.

Verification: The application executes business tasks through BookService, which safely triggers BookRepository without ever manually calling the new keyword inside the service class

Exercise 3 : Creating and Configuring a Maven Project

Core Objective: Covers project build automation, dependency mapping, and environment enforcement tools used in professional setups.

Key Project Milestones:

Artifact Blueprinting: Establishes a standard coordinate identity (groupId, artifactId, version) under the project title LibraryManagement.

Dependency Pipelines: Populates the pom.xml configuration with distinct library declarations for spring-context, spring-aop, and spring-webmvc to automate build downloads.

Compiler Compliance: Integrates the maven-compiler-plugin to force source and target configurations down to Java compliance version 1.8

Build Feature / AssetLocal Maven Workspace ImplementationHow It Relates to Your Code

Dependency Injection PipelineConfigured via pom.xml block declarationsPulls in the real Core container engines used to process setup tags automatically.

System Version EnforcementTarget compiler versions set strictly using build plugins (1.8)Ensures modern expressions don’t conflict with legacy client system installations.

Modular Framework LayersSplits source tracks into core business rules (WebMVC, AOP)Segregates background tracking operations clearly away from plain utility methods.

Spring Data JPA with Spring Boot, Hibernate

Spring Data JPA – Quick Example

1. Define the Entity (JPA/Hibernate)

This tells the application what the database table should look like.

```
Import jakarta.persistence.*;


@Entity
@Table(name = “customers”)
Public class Customer {

   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   Private Long id;


   @Column(nullable = false)
   Private String name;


   Private String email;


   // Getters, Setters, and Constructors
   Public Customer() {}
```

```
    Public Customer(String name, String email) {
        This.name = name;
        This.email = email;
    }

    Public Long getId() { return id; }
    Public String getName() { return name; }
    Public void setName(String name) { this.name = name; }
    Public String getEmail() { return email; }
    Public void setEmail(String email) { this.email = email; }
}
```

2. Create the Repository (Spring Data JPA)

By extending JpaRepository, Spring automatically provides methods like save(), findOne(), findAll(), and delete() without you writing any code

```
Import org.springframework.data.jpa.repository.JpaRepository;
Import java.util.List;

Public interface CustomerRepository extends
JpaRepository<Customer, Long> {
```

```
    // Spring Data JPA automatically generates the SQL for this based on the method name!
    List<Customer> findByEmail(String email);
}
```

3. Use it in your Service

Now you can inject the repository and start interacting with your database immediately

```
Import org.springframework.beans.factory.annotation.Autowired;
Import org.springframework.stereotype.Service;
Import java.util.List;

@Service
Public class CustomerService {

    @Autowired
    Private CustomerRepository customerRepository;

    Public void runQuickDemo() {
        // 1. Create and Save a new customer (Insert)
```

```
        Customer newCustomer = new Customer(“Alice Smith”, alice@example.com);
        customerRepository.save(newCustomer);

        // 2. Fetch all customers (Select * )
        List<Customer> allCustomers = customerRepository.findAll();

        // 3. Custom query executed automatically
        List<Customer> foundCustomers = customerRepository.findByEmail(“alice@example.com");
    }
}
```

Difference between JPA, Hibernate and Spring data JPA

The Difference: JPA vs. Hibernate vs. Spring Data JPA

To put it in perspective, think of it as a hierarchy of abstractions:

1. JPA (Jakarta Persistence API)

What it is: A specification (a set of guidelines, interfaces, and rules).

What it does: It defines how Object-Relational Mapping (ORM) should behave in Java, but it doesn't contain any actual running code to do the work. It's just the blueprint.

2. Hibernate

What it is: An implementation of the JPA specification.

What it does: It is the actual engine under the hood. Hibernate reads your JPA annotations (like @Entity) and does the heavy lifting of translating your Java objects into SQL queries and executing them against the database.

3. Spring Data JPA

What it is: An abstraction layer built on top of JPA (and usually Hibernate).

What it does: It eliminates boilerplate code. Instead of writing complex logic to open sessions, handle transactions, and write custom DAO (Data Access Object) implementations, Spring Data JPA allows you to just define an interface, and it generates the CRUD operations for you automatically.